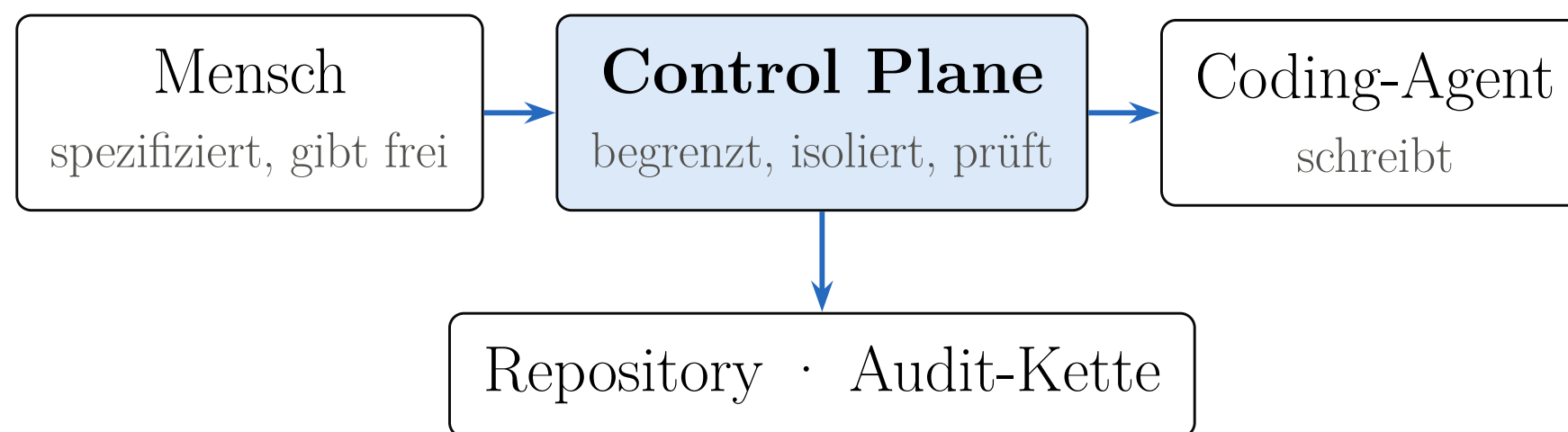


Der Engpass ist nicht das Modell. Es ist die fehlende Prozessschicht.

Coding-Agenten rufen Werkzeuge selbstständig auf. Das macht sie produktiv — und ihre Ausführung nichtdeterministisch und sicherheitsrelevant. Befunde der überarbeiteten Fassung, Stand August 2026.

Nicht möglichst viele Agenten. Eine Steuerschicht dazwischen.



Die Control Plane sitzt zwischen Mensch, Coding-Agent und Zielrepository. Sie begrenzt, isoliert, koordiniert, prüft und macht nachweisbar.

Ein Schreiber je Arbeitskopie. Beliebige viele Prüfer.

Zwei Agenten, die gleichzeitig dieselbe Arbeitskopie verändern, erzeugen Schreibkonkurrenz, unklare Zurechnung und Zwischenstände, für die niemand mehr geradestehen kann.

Wer schreibt, gibt nicht frei.

Erzeugung und Freigabe sind technisch getrennt. Read-only-Reviewer liefern strukturierte Befunde — und ein Agent bewertet nie allein die eigene Arbeit.

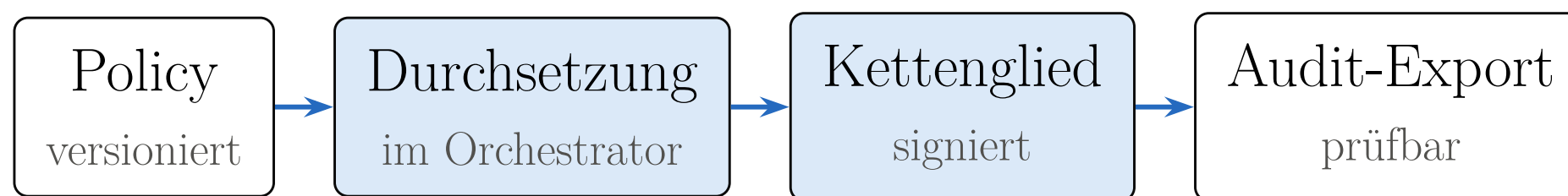
Ein Gate, dessen Ausfall wie Erfolg aussieht, ist schlimmer als kein Gate.

Ein ausgefallener Pflicht-Reviewer, eine nicht prüfbare Policy, eine fehlgeschlagene Attestierung: Nichts davon darf jemals als bestanden durchgehen.

Git trägt die Artefakte. Den Prozesszustand trägt es nicht.

Branches und Checkpoints gehören in Git. Workflow-, Policy-, Freigabe- und Audit-Zustand brauchen eine autoritative Quelle daneben — sonst ist der Prozess nicht rekonstruierbar.

Für jeden Lauf rekonstruierbar: welche Policy, welches Modell, welche Freigabe.



Jede Durchsetzung erzeugt ein signiertes Kettenglied in einer append-only Audit-Hashkette. Governance ist damit keine Ergänzung der Architektur, sondern Teil ihrer Struktur.

Kein Konzeptpapier: 28 fachliche Slices, 27 Releases in vier Monaten.

Rund 33.200 Zeilen Produktivcode, Testabdeckung als buildbrechendes Gate (Line ab 85 %, Branch ab 81 %). Die Architektur des Whitepapers war die Grundlage — das Kapitel beschreibt, was daraus geworden ist.
Stand: 6. August 2026, Release 0.20.0.

Parallelität entsteht aus Abhängigkeiten — nicht aus Agentenrollen.

Ein Feature wird in einen Task Graph zerlegt; unabhängige Tasks laufen als isolierte Child Runs, jeder mit eigenem Worktree und weiterhin mit genau einem Schreiber. Merge Coordinator und Integration Gate führen zusammen. Das Whitepaper trennt diesen Zielstand konsequent vom heute implementierten Stand.

Agentic Software Development

Enterprise Architecture with AI Agents — eine
Control-Plane-Architektur für den kontrollierten Einsatz von
Coding-Agenten

Vollständiges Whitepaper auf janda.io

Überarbeitete und erweiterte Fassung 2.1, Stand: August 2026.